Informe de la Aplicación Web

Mantenimiento Predictivo de Máquinas Industriales (AI4I 2020)

Proyecto 1 · Clasificación con Machine Learning y Aplicación Web
Inteligencia Artificial · Sexto Semestre · Ingeniería en Sistemas de Información · PUCE
Equipo: Alisson Basantes · David · Karol — Entrega: 11 de octubre de 2026

1. Introducción

El mantenimiento predictivo busca anticipar fallas en maquinaria industrial antes de que ocurran, reduciendo costos de reparación y tiempos de parada no planificados. Este proyecto aborda ese problema como una tarea de clasificación binaria sobre el dataset AI4I 2020 Predictive Maintenance, que registra 10.000 observaciones de condiciones de operación de máquinas (temperatura, velocidad rotacional, torque, desgaste de herramienta) junto con si la máquina falló o no. El trabajo cubre el ciclo completo: desde la exploración y preparación de los datos, pasando por el entrenamiento y comparación de varios algoritmos de clasificación, hasta el despliegue del modelo final en una aplicación web con una API REST documentada y protegida con autenticación.

2. Objetivos

2.1 Objetivo general

Desarrollar un sistema de clasificación basado en Machine Learning capaz de predecir fallas en máquinas industriales a partir de sus condiciones de operación, y desplegarlo en una aplicación web funcional que permita realizar predicciones en tiempo real.

2.2 Objetivos específicos

- Realizar un análisis exploratorio de datos (EDA) del dataset AI4I 2020 para identificar la distribución de la variable objetivo, el desbalance de clases y las variables más relevantes.
- Preparar y transformar los datos (codificación de la variable categórica "Type", escalado de variables numéricas, y exclusión de variables con fuga de información) para el entrenamiento de los modelos.
- Entrenar y comparar al menos dos algoritmos de clasificación (Regresión Logística, Random Forest, SVM y un modelo de ensamble) para identificar cuál tiene el mejor desempeño.
- Evaluar los modelos con métricas apropiadas al desbalance de clases (precisión, recall y F1-score de la clase minoritaria) y seleccionar el modelo final mediante ajuste de hiperparámetros con GridSearchCV.
- Diseñar e implementar una API REST, con un contrato definido previamente, autenticación JWT y documentación interactiva (Swagger/Redoc), que exponga el modelo entrenado.
- Construir una interfaz web que permita ingresar las variables de operación de una máquina y obtener una predicción en tiempo real junto con su probabilidad de falla.

3. Arquitectura de la aplicación

La aplicación sigue una arquitectura cliente-servidor con API REST, diseñada bajo un enfoque "contrato primero": el contrato (rutas, esquemas, códigos de respuesta y seguridad) se definió antes de implementar el código, documentado en docs/contrato_api.md.

Backend (FastAPI, Python): expone la API versionada en /api/v1, carga el modelo entrenado (Random Forest tuneado, exportado con joblib) y lo usa para generar predicciones. Incluye autenticación JWT (POST /auth/login), creación de predicciones (POST /predicciones, protegido), consulta del historial paginado (GET /predicciones, protegido) y un endpoint público de salud (GET /salud). Las respuestas siguen una estructura uniforme { status, message, data } en éxito y { status, error, details } en error, con los códigos HTTP correspondientes (200, 201, 400, 401).

Frontend (HTML, CSS y JavaScript puro): una página servida por el propio backend (mismo origen, sin problemas de CORS) con un formulario de login, un formulario de predicción con las seis variables de entrada, y una tabla que muestra el historial de predicciones realizadas, actualizada en tiempo real tras cada consulta a la API.

Documentación interactiva: generada automáticamente por FastAPI a partir del mismo contrato implementado, disponible en /docs (Swagger UI) y /redoc.

4. Integración del modelo

El modelo final (Random Forest con hiperparámetros ajustados mediante GridSearchCV) se exportó desde el notebook de entrenamiento con joblib y se incluye en el backend (backend/app/models/modelo_mantenimiento_predictivo.pkl). Al recibir una petición de predicción, el backend construye un DataFrame con las seis columnas que el modelo espera (Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min]), ejecuta el pipeline completo de preprocesamiento (codificación de "Type" y escalado de variables numéricas) ya incluido en el objeto exportado, y devuelve la clase predicha ("Falla" / "No falla") junto con la probabilidad de falla.

5. Resultados de los modelos evaluados

Comparación de los algoritmos entrenados sobre el conjunto de prueba, antes y después del ajuste de hiperparámetros. El modelo resaltado es el que se exportó y se usa en la aplicación:

Modelo	Accuracy	Recall falla	F1 falla
Regresión Logística	0.824	0.824	0.242
Random Forest (sin tuning)	0.980	0.471	0.615
SVM (sin tuning)	0.915	0.897	0.418
Ensamble RF+SVM (sin tuning)	0.980	0.515	0.642
Random Forest (tuneado) — modelo final	**0.982**	**0.691**	**0.723**
SVM (tuneado)	0.947	0.750	0.490
Ensamble RF+SVM (tuneado)	0.982	0.515	0.654

El Random Forest tuneado fue seleccionado como modelo final por tener el mejor F1 de la clase "falla" (0.723) manteniendo la accuracy más alta (0.982), con una mejora sustancial de recall respecto a su versión sin ajustar (de 0.471 a 0.691).

6. Capturas de pantalla del funcionamiento

Inicio de sesión con el usuario de demostración del equipo:

[ESPACIO PARA CAPTURA DE PANTALLA]
Captura 1 — Pantalla de login

Formulario de predicción con un caso real clasificado como "Falla":

[ESPACIO PARA CAPTURA DE PANTALLA]
Captura 2 — Predicción con resultado FALLA

Documentación interactiva de la API (Swagger UI):

[ESPACIO PARA CAPTURA DE PANTALLA]
Captura 3 — Swagger UI (/docs)

7. Instrucciones de instalación y ejecución local

1. Crear y activar un entorno virtual dentro de la carpeta backend/:

```
python -m venv venv  |  venv\Scripts\activate (Windows)
```

2. Instalar las dependencias:

```
pip install -r requirements.txt
```

3. Levantar el servidor:

```
uvicorn app.main:app --reload
```

4. Abrir http://localhost:8000 en el navegador e iniciar sesión con el usuario de demostración (usuario: equipo / contraseña: mantenimiento2026).

8. Limitaciones conocidas

- El historial de predicciones se guarda en memoria: se reinicia si se reinicia el servidor (no hay persistencia en base de datos).
- La aplicación corre en local; no está desplegada en un servidor accesible desde internet.
- La autenticación usa un único usuario de demostración fijo, sin un sistema de registro de usuarios ni almacenamiento seguro de contraseñas (adecuado para el alcance académico del proyecto, no para producción real).
- El modelo fue entrenado sobre datos sintéticos de un entorno controlado (AI4I 2020); su desempeño en datos de una planta real podría variar.

9. Conclusión

Se logró construir un pipeline completo de Machine Learning para mantenimiento predictivo, desde el análisis de un dataset con fuerte desbalance de clases (3.39% de fallas) hasta un modelo en producción. Tras comparar Regresión Logística, Random Forest, SVM y un ensamble de los dos últimos, el modelo con mejor desempeño fue un Random Forest con ajuste de hiperparámetros (F1 de la clase falla = 0.723, recall = 0.691, accuracy = 0.982), el cual se integró en una API REST con autenticación JWT y una interfaz web que permite predicciones en tiempo real. Como mejora futura, el equipo identifica: dar persistencia real al historial de predicciones, desplegar la aplicación en un servidor accesible fuera de la red local, y explorar si técnicas adicionales de balanceo de clases podrían mejorar aún más el recall sin sacrificar precisión.

10. Referencias

AI4I 2020 Predictive Maintenance Dataset [Dataset]. (2020). UCI Machine Learning Repository. https://doi.org/10.24432/C5HS5C

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. https://scikit-learn.org/

FastAPI. (s.f.). FastAPI Documentation. https://fastapi.tiangolo.com/